Rail Logistics Risk Analysis

U.S. Rail Equipment Accidents: Data Preparation, Dashboard Design & Findings | FRA Form 54 | 2024–2025

Oct 21, 2026 · @Aaron Obando

Overview

This document records how the U.S. rail accident dataset (FRA Form 54, Rail Equipment Accident/Incident reports) was cleaned, structured and turned into a five-page Power BI dashboard. It is Dashboard 4 of the semiconductor trade project and covers the logistics-risk objective of that project: understanding where and how the U.S. freight network breaks, and what it costs.

The analysis tests four hypotheses about severity, concentration, causes and geography. It does not measure semiconductor shipments or train-miles. It describes the risk context of the network that moves goods inland, which supports the nearshoring story (Mexico and the border states) and the concentration theme of the import analysis.

Dataset Summary

Attribute	Value
Source	FRA Form 54, Rail Equipment Accident/Incident reports
Time period	January 2024 – October 2025 (2025 covers 10 months)
Raw reports	4,333 rows, 115 columns
Columns kept	25 (90 dropped)
Unique events	3,554 (779 repeat or joint reports flagged)
Total damage cost	$1.02B across all reports, $907.6M across unique events
Median damage cost	$43.3K per unique event (mean $255K)
Deaths and injuries	90 deaths, 420 injuries (unique events)

Hypotheses and Purpose

Hypothesis	Test	Verdict
H1: Derailments drive logistics risk	Share of events vs share of cost by accident group	Confirmed: 55.6% of events, 78.9% of cost
H2: Cost is concentrated in a few events	Pareto curve of damage cost	Confirmed: top 1% = 39% of cost, top 10% = 74.9%
H3: Human errors are frequent but cheap, track and equipment are costly	Count share vs cost share and median cost by cause group	Mostly confirmed: human factors 35.2% of events, 17.3% of cost
H4: Border states carry extra risk	Border states (TX, NM, AZ, CA) vs rest of U.S.	Not confirmed: 19.3% of events, 17.2% of cost

Each hypothesis links to a theme of the semiconductor project. H1 and H3 ask what breaks. H2 uses the same concentration logic as the import analysis (HHI around 0.70). H4 uses the border states as a proxy for the U.S.–Mexico corridor in the nearshoring case.

Step 1 — Loading and Column Selection

Objective

Load only the columns needed for the four hypotheses and document everything that was dropped.

Code

```
import numpy as np, pandas as pd
from pathlib import Path

KEEP = ['Date', 'Time', 'Accident Type', 'Hazmat Released Cars',
        'State Abbreviation', 'State Name', 'County Name', 'Milepost',
'Latitude', 'Longitude',
        'Track Type', 'Track Class', 'Equipment Type',
        'Equipment Damage Cost', 'Track Damage Cost', 'Total Damage Cost',
        'Primary Accident Cause Code', 'Primary Accident Cause',
        'Total Persons Killed', 'Total Persons Injured',
        'Reporting Railroad Name', 'Other Railroad Name', 'Maintenance
Railroad Name',
        'Reporting Railroad Class', 'Narrative']

header = list(pd.read_csv(SRC, nrows=0).columns)        # read only the
header first
dropped = [c for c in header if c not in KEEP]
Path('dropped_columns.txt').write_text('\n'.join(dropped), encoding='utf-8')
df = pd.read_csv(SRC, usecols=[c for c in KEEP if c in header],
low_memory=False)
```

Key Design Choices

- **usecols as the drop:** pandas never loads the other 90 columns, so there is no drop list to maintain and the file loads lighter.
- **dropped_columns.txt:** the script writes the names of every dropped column, which documents the decision in the repository.
- **Time and Other Railroad Name** are kept only for duplicate detection and removed from the final fact table.
- **Original CSV, not Excel:** narratives contain line breaks. A round trip through Excel produced 4,515 rows, including 182 fragments with no date. The total cost still matched the original, so the valid rows were intact, but the script also drops any row without a date as a safety net.

Step 2 — Data Type Validation

Objective

Make sure money, count and date columns are numeric before any aggregation.

Code

```
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df = df.dropna(subset=['Date']).copy()

money = ['Equipment Damage Cost', 'Track Damage Cost', 'Total Damage Cost']
for c in money + ['Hazmat Released Cars', 'Total Persons Killed', 'Total
Persons Injured']:
    df[c] = pd.to_numeric(df[c].astype(str).str.replace(',', '').str.strip(),
errors='coerce')

df['Milepost'] = pd.to_numeric(df['Milepost'].astype(str).str.strip(),
errors='coerce')
df['Track Class'] = df['Track
Class'].astype(str).str.strip().str.upper().replace({'NAN': 'Unknown'})
df['State Name'] = df['State Name'].str.title()
df['Narrative'] = df['Narrative'].fillna('').str.replace(r'\s+', ' ',
regex=True).str.strip()
```

Key Design Choices

- **Costs arrive as text** with thousand separators (for example 20,000), so the commas are removed before converting. errors='coerce' turns anything unreadable into NaN instead of stopping the script.
- **Total Damage Cost is the headline metric.** In 1,660 rows it does not equal equipment plus track, because the total includes costs reported by other railroads. The components are kept but never added together.
- **Medians over means:** the median is $43.3K and the mean $255K, so a few extreme events distort any average.
- **Track Class** mixed X, x and O, so it was upper-cased and standardized.

Step 3 — Derived Fields

Objective

Turn hundreds of raw causes, accident types and equipment types into groups that a dashboard can show.

Code

```
df['Month'] = df['Date'].dt.to_period('M').dt.to_timestamp()

cause_map = {'H': 'Human factors', 'T': 'Track', 'E': 'Equipment',
             'S': 'Signal & communication', 'M': 'Miscellaneous'}
df['Cause Group'] = df['Primary Accident Cause
Code'].str[0].map(cause_map).fillna('Unknown')

def accident_group(t):
    if t == 'Derailment': return 'Derailment'
    if t in ('Hwy-rail crossing', 'RR grade crossing'): return 'Crossing'
    if 'collision' in str(t).lower(): return 'Collision'
    if t in ('Fire/violent rupture', 'Explosion-detonation'): return 'Fire /
explosion'
    return 'Other'
df['Accident Group'] = df['Accident Type'].map(accident_group)

def equipment_group(t):
    t = str(t)
    if t in ('Freight Train', 'Cut of cars', 'Single Car'): return 'Freight'
    if t == 'Yard/switching': return 'Yard / switching'
    if any(k in t for k in ('Passenger', 'Commuter', 'EMU')): return
'Passenger / commuter'
    return 'Other (MoW, light loco...)'
df['Equipment Group'] = df['Equipment Type'].map(equipment_group)

df['Border State'] = np.where(df['State Abbreviation'].isin({'TX', 'NM',
'AZ', 'CA'}),
                              'Border state (TX, NM, AZ, CA)', 'Rest of
U.S.')
df['Mainline'] = np.where(df['Track Type'] == 'Main', 'Main line', 'Yard /
industry / siding')

bins = [-1, 10_000, 50_000, 250_000, 1_000_000, np.inf]
labels = ['< $10K', '$10K-50K', '$50K-250K', '$250K-1M', '>= $1M']
df['Severity Bucket'] = pd.cut(df['Total Damage Cost'], bins=bins,
labels=labels).astype(str)
df['Mega Event (>= $10M)'] = np.where(df['Total Damage Cost'] >= 10_000_000,
'Yes', 'No')
```

Key Design Choices

- **Cause group from the first letter of the FRA cause code:** H human factors, T track, E equipment, S signal and communication, M miscellaneous.

- **Border states are a proxy** for the U.S.–Mexico corridor. California also includes Pacific ports, so the proxy is imperfect and is labeled as such.
- **Severity buckets and the $10M mega-event line** are analyst choices for readability, not official FRA categories.

Step 4 — Duplicate Reports

Objective

Find rows that describe the same incident without deleting any data.

Code

```
key = ['Date', 'Time', 'State Abbreviation', 'County Name', 'Milepost',
       'Accident Type', 'Total Damage Cost']
g = df.groupby(key, dropna=False)
df['Reports In Event'] = g['Date'].transform('size')
df['Railroads In Event'] = g['Reporting Railroad Name'].transform('nunique')
df['Is Primary Report'] = (~df.duplicated(key)).astype(int)
df['Duplicate Type'] = np.select(
    [df['Reports In Event'] == 1, df['Railroads In Event'] > 1],
    ['Single report', 'Joint report (several railroads)'],
    default='Repeat report (same railroad)')
```

Key Design Choices

- **Flag, do not delete.** Every row stays in the fact table, and Is Primary Report = 1 keeps one row per event, so Power BI can show unique events or all reports.
- **Time is part of the key.** Without it the key flags 794 rows instead of 779, which shows how much one column can change the result.

Key Insight

About 18% of the rows (779 of 4,333) were flagged: 630 repeat reports from the same railroad, mostly with identical narratives, and 149 joint reports from several railroads. Total cost drops from $1.02B to $907.6M once duplicates are removed, mainly because one $100.45M derailment in Mississippi was reported by two railroads. The reason for the repeat reports is not confirmed.

Step 5 — Hypothesis Tables and Exports

Objective

Create one small table per hypothesis, built from unique events only, to validate the dashboard.

Code

```
ev = df[df['Is Primary Report'] == 1].copy()

def summarize(frame, by):
    out = frame.groupby(by).agg(incidents=('Date', 'size'),
                                total_cost=('Total Damage Cost', 'sum'),
                                median_cost=('Total Damage Cost', 'median'),
                                killed=('Total Persons Killed', 'sum'),
                                injured=('Total Persons Injured',
'sum')).reset_index()
    out['share_of_incidents'] = out['incidents'] / frame.shape[0]
    out['share_of_cost'] = out['total_cost'] / frame['Total Damage
Cost'].sum()
    return out.sort_values('total_cost', ascending=False)

summarize(ev, 'Accident Group').to_csv(OUT /
'h1_severity_by_accident_group.csv', index=False)
summarize(ev, 'Cause Group').to_csv(OUT / 'h3_cause_groups.csv', index=False)
summarize(ev, 'Border State').to_csv(OUT / 'h4_border_vs_rest.csv',
index=False)

s = ev['Total Damage
Cost'].sort_values(ascending=False).reset_index(drop=True)
pareto = pd.DataFrame({'rank_pct': (s.index + 1) / len(s), 'cum_cost_share':
s.cumsum() / s.sum()})
pareto.iloc[::max(1, len(pareto) // 200)].to_csv(OUT / 'h2_pareto_curve.csv',
index=False)
```

Key Design Choices

- **Unique events only** for every hypothesis table, so duplicates cannot inflate shares.
- **Shares and medians, not averages,** so the extreme events do not dominate the comparison.
- **The Pareto curve is sampled to about 200 points** so it loads fast and still draws a smooth line.

Step 6 — Power BI Model and Measures

Model

- **fact_incidents** (4,333 rows) related to **dim_date** on Date (one to many).
- **h2_pareto_curve, h2_concentration_summary and qa_log** stay disconnected and feed only the Pareto line, its two KPI cards and the Method page.
- **Pages 1 to 4 carry a page-level filter Is Primary Report = 1.** Page 5 has no filter so it can show the duplicates.

Measures

```
Unique Events = COUNTROWS(fact_incidents)
Total Damage Cost = SUM(fact_incidents[Total Damage Cost])
Median Cost = MEDIAN(fact_incidents[Total Damage Cost])
Deaths = SUM(fact_incidents[Total Persons Killed])
Injuries = SUM(fact_incidents[Total Persons Injured])
Event Share = DIVIDE([Unique Events], CALCULATE([Unique Events],
ALLSELECTED(fact_incidents)))
Cost Share = DIVIDE([Total Damage Cost], CALCULATE([Total Damage Cost],
ALLSELECTED(fact_incidents)))
Median Border = CALCULATE([Median Cost], fact_incidents[Border State] =
"Border state (TX, NM, AZ, CA)")
Median Rest = CALCULATE([Median Cost], fact_incidents[Border State] = "Rest
of U.S.")
Repeat Reports = CALCULATE(COUNTROWS(fact_incidents), fact_incidents[Is
Primary Report] = 0, fact_incidents[Duplicate Type] = "Repeat report (same
railroad)")
Joint Reports = CALCULATE(COUNTROWS(fact_incidents), fact_incidents[Is
Primary Report] = 0, fact_incidents[Duplicate Type] = "Joint report (several
railroads)")
```

Key Design Choices

- **ALLSELECTED in the share measures** makes shares respond to slicers while staying a share of what is currently shown.
- **Severity Order column** is built from Total Damage Cost, not from Severity Bucket. Building it from the bucket column created a circular dependency when used for sorting.

Step 7 — Dashboard Design

Page	Headline	Main visuals	Purpose
1. Executive Summary	Most rail incidents are routine, but a few derailments drive outsized losses	Five KPI cards, four verdict cards (H1 to H4), monthly trend, key takeaway	Give the whole story in one screen
2. Failure Drivers	Equipment and track failures lead costs, while crossings dominate accident impacts	Median cost by cause, event vs cost by main line and yard, event vs cost by accident type, crossing fatalities	Test H1 and H3
3. Cost Concentration	The top 10% of incidents generate 75% of total damage costs	Pareto curve, top 1% and top 10% cards, events and cost by severity bucket, mega-event slicer	Test H2
4. Geographic Concentration	Texas leads incident volume, while the top 10 states account for about half of all reports	Border vs rest median cards, state map, top 10 states	Test H4
5. Methodology & Limitations	How rail incident records were cleaned, transformed and analyzed	Report, duplicate and cost cards, process flow, data preparation, limitations, GitHub link	Make the analysis auditable

Key Design Choices

- **Insight headlines:** every page title states the finding, not just the topic.
- **Consistent colors:** blue for events, orange for cost, on every page.
- **Disclaimer:** the dashboard states that the data comes from FRA Form 54 and that the project is independent and not affiliated with the U.S. DOT or FRA.

Quality Assurance

The dashboard was reconciled against the Python tables before it was finalized.

Check	Expected value
Unique events	3,554
Total damage cost	$907.6M
Median damage cost	$43.3K
Deaths / injuries	90 / 420
Derailment event share / cost share	55.6% / 78.9%
Top 10% cost share	74.9%
Border event share / cost share	19.3% / 17.2%

Issues found during review and fixed:

- **Duplicates counted on a page:** crossing deaths read 98 instead of 78 until the Is Primary Report page filter was applied.
- **Pareto curve flattened near 0%:** the visual was showing each point as a percent of the column total. Setting Show value as to No calculation fixed it.
- **Top states chart ranked by name,** not by events, so it listed Vermont and Washington instead of Illinois and California. The Top N filter was changed to rank by Unique Events.
- **Median cards swapped:** border states are $46.62K and the rest of the U.S. is $42.66K.
- **Monthly trend grouped by month name** created a false drop in November and December, because 2025 stops in October. The axis was changed to a continuous month-start date.
- **A 100% stacked chart** distorted event and cost shares, and was replaced by a clustered column chart.

Summary of Findings

Finding #1 — Derailments Drive Cost

Derailments are 55.6% of unique events but 78.9% of damage cost, with a median of $56.0K per event against $26.9K for crossings. Crossings behave differently: they caused 78 of the 90 deaths, while derailments caused none. Derailments cost money, crossings cost lives. Main-line track holds 39.3% of events but 74.9% of cost, and freight equipment is 50.1% of events and 67.6% of cost.

Finding #2 — Cost Is Highly Concentrated

The top 1% of events account for 39% of damage cost, the top 5% for 63.4% and the top 10% for 74.9%. The median event costs $43.3K while the mean is $255K, which is why medians are used throughout.

Finding #3 — Human Errors Are Frequent but Cheap

Human factors are 35.2% of events but 17.3% of cost, with a median of $42.3K. Track and equipment causes are less frequent but costlier per event, with medians of $64.3K and $64.6K. The Miscellaneous group (45.7% of cost) is distorted by the $100M Mississippi event and by causes still under investigation.

Finding #4 — No Excess Severity in Border States

Border states hold 19.3% of events and 17.2% of cost, with a median of $46.62K against $42.66K for the rest of the country. That is no evidence of excess severity. Texas leads with 394 events (11.1%), followed by Illinois (236) and California (221), and the top 10 states account for 49% of events. Counts partly reflect network size.

How This Complements the Semiconductor Project

- **It adds the logistics-risk component** named in the project objectives, which the trade, tariff and workforce dashboards did not cover.
- **The concentration logic matches the import analysis:** a few countries carry most import value, and a few events carry most rail damage cost.
- **The border-state view connects to the nearshoring case,** where Mexico's USMCA advantage depends on the inland network that carries the goods.
- **It does not link semiconductor shipments to rail incidents.** It describes the network, not the cargo.

Analysis Limitations

- There is no train-mile or traffic data, so the analysis shows counts and costs, not risk rates, and it is not a ranking of states or railroads.
- Only reported incidents are included, and reports may be subject to a damage threshold that has not been confirmed.
- Repeat and joint reports can overlap, and the reason for repeat reports is not confirmed.
- Damage costs are railroad-reported estimates, and a few extreme events weigh heavily on totals.

- Coverage is January 2024 to October 2025, so 2025 has 10 months and no seasonality can be established.
- Some fields are incomplete, such as causes still under investigation.

Files Generated During This Analysis

File	Description
rail_pipeline.py	Cleaning, derived fields, duplicate flags and table exports
powerbi_tables/fact_incidents.csv	Fact table, one row per report, with duplicate flags
powerbi_tables/dim_date.csv	Calendar table for time intelligence
powerbi_tables/h1 to h4 tables	One summary table per hypothesis
powerbi_tables/ctx tables	Context tables by equipment group, main line vs yard, and month
powerbi_tables/qa_log.csv	Row counts, duplicate counts and cost totals
powerbi_tables/dropped_columns.txt	The 90 columns that were not loaded
rail-logistics-risk-dashboard.pbix	The five-page Power BI report

Document Metadata

- Data Source: FRA Form 54, Rail Equipment Accident/Incident reports
- Period: January 2024 – October 2025
- Analysis Tools: Python (pandas, numpy), Power BI (DAX)
- Output Files: rail_pipeline.py, powerbi_tables, rail-logistics-risk-dashboard.pbix
- Author: Aaron Obando Gomez